

VINH UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY



COURSE PROJECT REPORT

Subject: Software Technology

DESIGN AND DEVELOPMENT OF A SAAS-BASED FOOTBALL FIELD MANAGEMENT SYSTEM ON WEBSITE AND MOBILE APPLICATION PLATFORMS

Nghe An, 06/2026

Performed by: TRAN DUC LUONG

Student ID: 225748020110186

Major: Information Technology

Specialization: Software Engineering

Supervisor: Ph.D. Cao Thanh Son

COURSE PROJECT TOPIC

Full name: Tran Duc Luong

Student ID: 225748020110186

Cohort: 63

School: School of Engineering and Technology

1. Topic

Design and Development of a SaaS-Based Football Field Management System on Website and Mobile Application Platforms

2. Content

Researching and developing a SaaS-based football field management system on website and mobile application platforms, integrating AI and payment gateways.

3. Topic assignment date: March 2026

4. Topic complete date: May 2026

May 2026

INSTRUCTOR

Ph.D Cao Thanh Son

PREFACE

First and foremost, I would like to express my sincere gratitude to my supervisor, whose expertise, guidance, and continuous support have been invaluable throughout the development of this thesis project.

I am also deeply thankful to the faculty members of the School of Engineering and Technology at Vinh University for providing the foundational knowledge and academic environment necessary to undertake this endeavor.

Finally, I wish to extend my appreciation to my family and friends for their unwavering encouragement. This thesis, "Building a Multi-Tenant SaaS Platform for Football Field Management integrating AI and VNPay", represents the culmination of my academic journey and a stepping stone into the professional software engineering world.

TABLE OF CONTENTS

ABBREVIATIONS	x
LIST OF FIGURES	xi
LIST OF TABLES	xii
ABSTRACT	xiii
DUTY ROSTER	xiv
CHAPTER 1: INTRODUCTION TO THE PROBLEM	1
1.1 Background and Motivation	1
1.2 Competitive Analysis	1
1.3 Problem Statement	2
1.4 Objectives of the Study	3
1.5 Scope and Limitations	3
1.6 Project Deliverables	4
1.7 Research Methodology	4
1.8 Project Organization and Management	5
1.9 Risk Management	5

1.10	Expected Contributions	6
CHAPTER 2: SYSTEM ANALYSIS AND DESIGN		7
2.1	Software as a Service (SaaS) and Multi-Tenant Architecture	7
2.2	Backend Technologies: Node.js and Express.js	8
2.3	Relational Databases and Query Builders	8
2.4	Artificial Intelligence and Function Calling Mechanics	9
2.5	Cross-Platform Mobile Development with Capacitor	10
2.6	Cloud Storage and Persistent Media Management	10
2.7	Real-time Alerts via Firebase Cloud Messaging (FCM)	11
2.8	Native Printing and Hardware Bridging	11
2.8.1	User Persona: The Modern Facility Owner	11
2.8.2	User Persona: The Tech-Savvy Athlete	12
2.8.3	User Persona: The Field Staff	12
2.9	Requirements Analysis	12
2.9.1	Non-Functional Requirements	13
2.10	Use Case Scenarios	15
2.10.1	Actor Definitions	15
2.10.2	Extended Functional Requirements	15
2.10.3	Detailed Use Case: AI-Assisted Booking	17

2.10.4	Use Case: Management of Sport Facilities	18
2.10.5	Use Case: Financial Reporting and Analytics	19
2.11	Detailed UI/UX Design Principles	19
2.11.1	Color Palette and Typography	19
2.11.2	Interface Layouts	20
2.11.3	Detailed Use Case: QR Code Check-in	20
2.12	Database Design and ERD	21
2.12.1	Detailed Data Dictionary	21
2.13	System Activity and Workflows	24
2.13.1	Workflow: Tenant Onboarding and Setup	24
2.13.2	Workflow: Booking Cancellation and Refund	26
2.14	System Sequence Diagrams	26
CHAPTER 3: BUILD THE SYSTEM		27
3.1	Backend Architecture Implementation	27
3.1.1	Middleware and Security Patterns	29
3.1.2	Real-time Monitoring: WebRTC and Socket.io	29
3.2	AI Integration: Implementing Function Calling	30
3.2.1	AI Context Injection and Prompt Engineering	32
3.3	Modular Design System and Visual Aesthetics	32

3.4	Mobile App Implementation: Capacitor Hardware Bridge	33
3.5	System Testing and Quality Assurance	34
3.5.1	Unit Testing: Logic Verification	34
3.5.2	Integration Testing: End-to-End Workflows	34
3.5.3	User Acceptance Testing (UAT)	35
3.6	Cloud Deployment Architecture	35
3.6.1	Database Optimization and Indexing	36
3.6.2	Handling Media with Cloudinary	37
3.6.3	Security Hardening	37
3.7	Evaluation of Results	37
3.8	Limitations	38
3.9	Future Work	38
3.10	Future Development Roadmap	39
3.10.1	Social and Community Features	39
3.10.2	Enterprise Operations	39
3.11	System Maintenance and Backup Procedures	40
3.11.1	Data Backup Strategy	40
3.11.2	Monitoring and Alerting	40
3.12	Conclusion	40

REFERENCES	41
APPENDIX	42
3.12.1 Provider Admin: Managing the Ecosystem	44
3.12.2 Tenant Admin: Daily Facility Operations	44
3.12.3 Customer: Finding and Booking a Match	45

ABBREVIATIONS

SaaS	Software as a Service
PaaS	Platform as a Service
UI/UX	User Interface / User Experience
API	Application Programming Interface
FCM	Firebase Cloud Messaging
ERD	Entity-Relationship Diagram
RDBMS	Relational Database Management System
SDK	Software Development Kit
LLM	Large Language Model
MVC	Model-View-Controller
JWT	JSON Web Token
SQL	Structured Query Language
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
WebRTC	Web Real-Time Communication
QR	Quick Response (Code)
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
DBaaS	Database as a Service
CI/CD	Continuous Integration / Continuous Deployment

LIST OF FIGURES

Figure 2.1	General Use Case Diagram	16
Figure 2.2	Entity-Relationship Diagram	22

Figure 2.3 AI Booking Sequence 27

Figure 3.1 Full-stack Multi-tenant SaaS Architecture with External Integrations 28

Figure 3.2 UML State Machine: The Booking Lifecycle and State Transitions 31

LIST OF TABLES

Table 1.1 Comparison Matrix: FootField vs Market Alternatives 2

Table 1.2 System Development Requirements and Tech Stack 5

Table 1.3 Project Risk Assessment and Mitigation Matrix 6

Table 2.1 FootField SaaS Platform: Subscription Tiers and Features 14

Table 2.2 Actor-Role Permission Matrix 14

Table 2.3 Detailed Functional Requirements and Prioritization 17

Table 2.4 AI Service: Tool-Calling Schema Definitions 21

Table 2.5 Data Dictionary: Tenants Table 22

Table 2.6 Data Dictionary: Fields Table 23

Table 2.7 Data Dictionary: Subscription Packages 23

Table 2.8 Data Dictionary: Financial Transactions 24

Table 2.9 Data Dictionary: Bookings Table 25

Table 3.1 Core Backend API Endpoints and Access Control Policies 30

Table 3.2 Unit Test Cases for Backend Logic 34

Table 3.3 Integration Test Cases for System Workflows 35

Table 3.4 User Acceptance Testing Results across Platforms 35

Table 3.5 Strategic Indexing Implementation for Scalability 36

Table 3.6 Comprehensive REST API Endpoint Specification 46

ABSTRACT

This project presents FootField, a comprehensive B2B2C Software as a Service (SaaS) platform designed to digitize the management and booking operations of football facilities. The system bridges the gap between facility owners (Tenants), platform administrators (Providers), and end-users (Customers) within a unified ecosystem. Built on a robust Node.js backend with a multi-tenant MySQL architecture, FootField leverages Google Gemini 2.5 Flash for conversational AI booking via function calling, and utilizes Firebase Cloud Messaging (FCM) for real-time notifications. The frontend follows a modular design system for high performance, while the mobile applications, powered by Ionic Capacitor, integrate native hardware features such as QR code scanning and wireless printing. This hybrid approach ensures high scalability, visual consistency, and a seamless cross-platform user experience.

TÓM TẮT

Dự án này giới thiệu FootField, một nền tảng Phần mềm dưới dạng dịch vụ (SaaS) mô hình B2B2C toàn diện, được thiết kế để số hóa các hoạt động quản lý và đặt sân của các cơ sở bóng đá. Hệ thống kết nối khoảng cách giữa chủ sở hữu cơ sở (Tenants), quản trị viên nền tảng (Providers) và người dùng cuối (Customers) trong một hệ sinh thái thống nhất. Được xây dựng trên nền tảng Node.js mạnh mẽ với kiến trúc MySQL đa người thuê (multi-tenant), FootField tận dụng Google Gemini 2.5 Flash cho tính năng đặt sân qua trí tuệ nhân tạo (AI) đàm thoại thông qua cơ chế function calling, đồng thời sử dụng Firebase Cloud Messaging (FCM) cho các thông báo thời gian thực. Giao diện người dùng (frontend) tuân theo hệ thống thiết kế mô-đun để đạt hiệu suất cao, trong khi các ứng dụng di động, được hỗ trợ bởi Ionic Capacitor, tích hợp các tính năng phần cứng

gốc như quét mã QR và in ấn không dây. Cách tiếp cận kết hợp này đảm bảo khả năng mở rộng cao, tính nhất quán về mặt hình ảnh và trải nghiệm người dùng liền mạch trên nhiều nền tảng.

Keywords: SaaS, Multi-tenancy, Conversational AI, Function Calling, Node.js, Capacitor, FCM.

DUTY ROSTER

No.	Job	Name
1	System Analysis & Requirements	Tran Duc Luong
2	Database Design & Implementation	Tran Duc Luong
3	Backend API Development	Tran Duc Luong
4	Frontend UI/UX Development	Tran Duc Luong
5	AI Chatbot Integration	Tran Duc Luong
6	VNPay Payment Integration	Tran Duc Luong
7	Mobile App Packaging	Tran Duc Luong
8	Deployment & Testing	Tran Duc Luong
9	Documentation	Tran Duc Luong

CHAPTER 1: INTRODUCTION TO THE PROBLEM

1.1 Background and Motivation

In recent years, the sports and recreation industry, particularly the management of artificial football fields, has experienced significant growth. With increasing awareness of health and fitness, the demand for accessible sports facilities has surged. In urban areas, artificial football fields serve as primary venues for amateur leagues, corporate team-building events, and casual recreational matches.

Despite this rapid expansion in demand, the operational management of these facilities remains predominantly traditional and localized. Facility owners often operate independently, lacking standardized software to manage their daily workflows. The reliance on manual processes not only increases the administrative burden but also limits the scalability of the business. The motivation for this project stems from the urgent need to bridge the gap between traditional sports facility management and modern cloud-based software solutions, thereby democratizing access to enterprise-grade management tools for small and medium-sized sports facility owners.

1.2 Competitive Analysis

To understand the unique value proposition of FootField, it is necessary to compare it with the current market alternatives available to sport facility owners.

Traditional physical ledgers offer zero data visibility and are prone to loss or damage. Generic ERP systems, while powerful, are often too complex and expensive for small facility owners. FootField sits in the "sweet spot" by providing industry-specific tools (like the booking grid and AI assistant) with the low-cost benefits of a multi-tenant SaaS model.

Feature	Traditional Logs	Generic ERP	FootField (SaaS)
Multi-tenant	No	Partial	Full Native
AI Assistant	No	No	24/7 Gemini Bot
Mobile Check-in	No	Complex	Native QR App
Payment Sync	Manual	Semi-auto	Full VNPAY Sync
Setup Cost	Low	High (Custom)	Low (Subscription)

Table 1.1 Comparison Matrix: FootField vs Market Alternatives

1.3 Problem Statement

The current landscape of football field management is plagued by several critical inefficiencies:

- **Manual Bookkeeping and Scheduling Errors:** Facility owners typically rely on physical ledgers or disjointed spreadsheet software. This manual approach is highly susceptible to human error, leading to overlapping bookings (double-booking) and lost revenue.
- **Poor Customer Experience:** Customers must manually call or message facility managers to inquire about field availability. This process is time-consuming, lacks transparency, and heavily depends on the manager's availability to respond.
- **Inefficient Payment and Verification:** Payments are mostly handled via cash or manual bank transfers without automated reconciliation. Verifying whether a customer has paid or arrived at the field is a tedious process for the staff.
- **Lack of Scalability:** Developing a custom software application for a single facility is financially unviable for most owners. The high costs of software development, server maintenance, and IT support act as significant barriers to digital transformation.

1.4 Objectives of the Study

To address the aforementioned problems, this project aims to design and develop "FootField," a comprehensive, cloud-based platform. The specific objectives are as follows:

1. **Develop a Multi-Tenant SaaS Platform:** Create a single, unified software ecosystem capable of securely hosting and managing multiple independent football facility businesses simultaneously, drastically reducing the cost of software distribution.
2. **Automate Booking and Scheduling:** Implement a robust scheduling engine with an intuitive time-grid interface to eliminate double-booking and streamline daily operations for facility managers.
3. **Integrate Artificial Intelligence:** Deploy a Large Language Model (LLM) powered chatbot to serve as a 24/7 virtual assistant, enabling customers to query availability and execute bookings using natural language.
4. **Digitize Check-in Processes:** Develop a cross-platform mobile application that leverages the device's native camera for rapid QR code scanning, allowing on-site staff to verify electronic tickets instantly.
5. **Streamline Financial Transactions:** Integrate a secure online payment gateway (VNPay) to automate transaction verification and invoice generation.

1.5 Scope and Limitations

The scope of this project encompasses the development of three interconnected modules: the Provider Admin dashboard (for SaaS ecosystem management), the Tenant Admin dashboard (for facility owners), and the Customer Storefront (for end-users). The system supports core functionalities including field management, booking workflows, customer relationship management (CRM), and financial reporting.

However, the project is subject to certain limitations. Currently, the AI chatbot is dependent on the Google Gemini API, which introduces external network latency and relies on a constant internet connection. Furthermore, the payment integration is strictly limited to the VNPay Sandbox environment for demonstration purposes, and the system does not yet support hardware integrations such as IoT-enabled smart lighting for the football fields.

1.6 Project Deliverables

The project results in a suite of integrated software components, each serving a specific role in the ecosystem:

1. **Backend Core (Node.js):** A high-performance API server with integrated multi-tenant security and external service handlers (AI, Payment).
2. **Provider Management Portal (Web):** An administrative dashboard for the platform owner to manage global settings and tenant accounts.
3. **Tenant Business Portal (Web):** A comprehensive ERP-lite system for football field owners to manage their operations.
4. **Customer Storefront (Responsive Web):** A public-facing booking engine optimized for both desktop and mobile browsers.
5. **Staff Utility App (Android):** A native mobile application built with Capacitor for on-field operations like scanning and printing.
6. **Technical Documentation:** This comprehensive thesis document and the associated source code documentation.

1.7 Research Methodology

The development of the FootField platform follows the Agile Software Development Life Cycle (SDLC). This methodology allows for iterative development, continu-

ous testing, and rapid adaptation to changing requirements. The project follows a structured agile methodology, divided into key sprints as shown in the table below:

No.	Component	Specification / Technology
1	Runtime Environment	Node.js v18+ (LTS)
2	Backend Framework	Express.js 4.x
3	Database Engine	MySQL 8.0 (Relational)
4	AI Integration	Google Gemini 1.5 Flash (LLM)
5	Mobile Bridge	Ionic Capacitor (Android Native)
6	Payment Gateway	VNPay Secure Sandbox
7	Cloud Deployment	Render (PaaS) & Aiven (DBaaS)

Table 1.2 System Development Requirements and Tech Stack

1.8 Project Organization and Management

The FootField project was managed using the Scrum framework, an iterative and incremental Agile methodology. The project was organized into five distinct sprints, each with its own set of goals and deliverables.

- **Product Owner:** Responsible for defining the product vision and prioritizing the product backlog.
- **Scrum Master:** Ensures that the team adheres to Scrum values and practices, facilitating sprint planning and daily stand-ups.
- **Development Team:** Responsible for the technical implementation of the platform, including backend, frontend, and mobile components.

1.9 Risk Management

Developing a complex SaaS platform involving AI and financial transactions involves several risks. The following mitigation strategies were implemented:

Risk Category	Potential Impact	Mitigation Strategy
Data Breach	Unauthorized access to sensitive customer/tenant data.	Implementation of strict multi-tenant logic and row-level isolation.
API Failure	Dependence on external Gemini or VNPAY APIs.	Implementation of robust error handling and fallback mechanisms.
Scalability	System slowdown during peak booking hours.	Use of Aiven's managed DB with auto-scaling and Render's load balancing.
Mobile Compatibility	Issues on specific Android hardware.	Use of Capacitor's standardized bridge and cross-device testing.

Table 1.3 Project Risk Assessment and Mitigation Matrix

1.10 Expected Contributions

The successful completion of the FootField platform is expected to yield several significant contributions to both the academic field and the sports management industry:

- **Technical Contribution:** Demonstrating a viable architecture for multi-tenant SaaS applications using open-source technologies like Node.js and MySQL. The project provides a blueprint for integrating conversational AI into deterministic business workflows via LLM function calling.
- **Economic Contribution:** By lowering the barrier to entry for digital transformation, FootField enables small football facility owners to compete with larger franchises through enterprise-grade management tools at a fraction of the cost.
- **Operational Contribution:** Automating the booking and check-in lifecycle reduces administrative overhead and minimizes human error, directly leading to improved revenue consistency and customer satisfaction.

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN

This chapter translates the conceptual goals of the FootField platform into concrete engineering specifications. It details the functional requirements, identifies system actors, presents use case scenarios, and outlines the architectural and database designs. Additionally, it provides an in-depth review of the underlying technologies and architectural paradigms that form the foundation of the FootField platform.

2.1 Software as a Service (SaaS) and Multi-Tenant Architecture

Software as a Service (SaaS) is a cloud computing model where software applications are hosted by a third-party provider and made available to customers over the internet. Unlike the traditional on-premises software model, SaaS eliminates the need for organizations to install and run applications on their own computers or data centers, thereby reducing hardware acquisition, provisioning, and maintenance costs [1].

A fundamental characteristic of a highly scalable SaaS application is its **Multi-Tenant Architecture**. In a multi-tenant environment, a single instance of the software and its underlying database serves multiple distinct customer organizations (tenants). There are several approaches to implementing multi-tenancy at the database level:

- **Separate Databases:** Each tenant is allocated a completely isolated database. This provides the highest level of data security but incurs immense infrastructure costs and makes schema updates extremely difficult to manage across thousands of databases.
- **Shared Database, Separate Schemas:** All tenants share the same database instance, but each has its own schema (a distinct set of tables). This offers a middle ground between isolation and resource sharing.
- **Shared Database, Shared Schema:** All tenants share the exact same database

and the same tables. Data isolation is achieved logically at the application level by associating every row of data with a unique identifier, commonly referred to as a `tenant_id`.

The FootField platform adopts the **Shared Database, Shared Schema** approach. This method was selected for its exceptional cost-efficiency and ease of maintenance. Every query executed by the backend must be strictly scoped using the `tenant_id` to guarantee that tenants cannot access or modify each other's data.

2.2 Backend Technologies: Node.js and Express.js

Node.js is an asynchronous, event-driven JavaScript runtime built on Chrome's V8 JavaScript engine [2]. Traditional web servers, such as Apache, handle concurrent requests by spawning a new thread for each connection, which consumes significant memory. In contrast, Node.js operates on a single-threaded event loop. When it encounters an I/O operation (such as querying a database), it delegates the task to the operating system and continues processing other requests. Once the I/O operation completes, a callback function is placed in the event queue to be executed. This non-blocking architecture makes Node.js highly scalable and exceptionally well-suited for data-intensive, real-time applications like booking systems.

Express.js is a minimal and flexible web application framework running on top of Node.js [3]. It provides a robust set of features for routing HTTP requests, handling middleware, and serving static files. In this project, Express.js acts as the backbone of the Monolithic Hybrid architecture, managing RESTful API endpoints while concurrently serving the frontend HTML, CSS, and client-side JavaScript assets.

2.3 Relational Databases and Query Builders

For structured financial and operational data, relational database management systems (RDBMS) are the industry standard. MySQL, an open-source RDBMS, was chosen

for its reliability, ACID (Atomicity, Consistency, Isolation, Durability) compliance, and widespread community support [4].

To interact with MySQL securely and efficiently from within the Node.js environment, the project utilizes Knex.js [5]. Knex.js is a "batteries-included" SQL query builder. Unlike Object-Relational Mappers (ORMs) which can sometimes obscure the underlying SQL and lead to performance bottlenecks, a query builder allows developers to write SQL-like syntax in JavaScript. Crucially, Knex.js automatically parametrizes queries, which is the primary defense mechanism against SQL Injection vulnerabilities. It also provides a robust migration system, allowing developers to version-control changes to the database schema.

2.4 Artificial Intelligence and Function Calling Mechanics

The integration of Artificial Intelligence transforms the platform from a passive tool into an active assistant. The platform leverages Google Gemini 2.5 Flash, an advanced Large Language Model (LLM) [6].

While traditional chatbots rely on rigid, pre-programmed decision trees (e.g., "Press 1 for Booking, Press 2 for Support"), LLMs utilize Natural Language Processing (NLP) to understand the semantic intent behind a user's unstructured input. However, an LLM on its own is a "closed system"; it can generate text but cannot interact with external databases.

To overcome this, the project utilizes a relatively new paradigm known as **Function Calling** (or Tool Use). The workflow is as follows:

1. **Definition:** The backend developer defines a set of functions (e.g., `check_availability`, `create_booking`) and provides the LLM with a JSON schema describing what these functions do and what parameters they require.
2. **Inference:** The user sends a message (e.g., "Book a 7-a-side field for tomorrow at

18:00"). The LLM analyzes the text, recognizes that the user wants to book a field, and instead of replying with text, it outputs a structured JSON object requesting the backend to execute the `create_booking` function with the extracted parameters.

3. **Execution:** The Node.js backend intercepts this request, executes the corresponding SQL query via Knex.js, and obtains the result (e.g., success or failure).
4. **Resolution:** The backend sends the result back to the LLM. The LLM then synthesizes a natural language response (e.g., "I have successfully booked the 7-a-side field for you!") and presents it to the user.

This mechanism bridges the gap between conversational AI and deterministic software execution.

2.5 Cross-Platform Mobile Development with Capacitor

Ionic Capacitor solves this problem by providing a cross-platform native runtime [7]. It allows developers to build mobile applications using standard web technologies (HTML, CSS, JavaScript). Capacitor acts as a bridge; it wraps the web application in a native WebView and exposes a set of JavaScript APIs that can communicate with native device hardware. In the context of the FootField platform, Capacitor is used to grant the web-based Tenant Admin dashboard access to the device's native camera, enabling the implementation of a high-speed QR code scanner for customer check-ins without the need for native code development.

2.6 Cloud Storage and Persistent Media Management

Deploying applications on Platform-as-a-Service (PaaS) providers like Render often involves working with ephemeral file systems. To achieve data persistence, the platform integrates with Cloudinary, a cloud-based media management service. By using the Cloudinary SDK, the backend automatically uploads files to the cloud and stores only the permanent URL in the MySQL database, ensuring that media assets remain

accessible indefinitely.

2.7 Real-time Alerts via Firebase Cloud Messaging (FCM)

To ensure facility staff are immediately notified of new bookings, the platform utilizes Firebase Cloud Messaging (FCM). The backend utilizes the Firebase Admin SDK to trigger push notifications based on specific system events (e.g., successful payment). These notifications are delivered to the mobile application via a persistent connection, providing the "real-time" responsiveness required for a commercial service.

2.8 Native Printing and Hardware Bridging

The platform leverages the `cordova-plugin-printer` within the Capacitor environment to establish a bridge between the web interface and the mobile operating system's printing services. This allows the Tenant Admin dashboard to trigger native print dialogs and communicate with Bluetooth or Wi-Fi thermal printers directly from the mobile application.

The platform is designed to serve three distinct primary actors, each with specific privileges and functional needs. To better understand these requirements, we define the following user personas:

2.8.1 User Persona: The Modern Facility Owner

Name: Mr. Binh

Goal: Wants to reduce the time spent answering phone calls and managing a paper diary. He values real-time revenue visibility and a professional digital presence for his field business.

Pain Point: Double-bookings and "no-shows" that cause financial loss.

2.8.2 User Persona: The Tech-Savvy Athlete

Name: Tung

Goal: Wants to book a match quickly via his smartphone without talking to anyone.

Prefers digital payments and needs instant confirmation.

Pain Point: Having to call multiple fields just to find an empty slot at 7 PM.

2.8.3 User Persona: The Field Staff

Name: Hoa

Goal: Needs a fast way to verify who has paid and who hasn't. Needs to check-in players quickly as they arrive.

Pain Point: Argumentative customers who claim they paid when the system shows they haven't.

2.9 Requirements Analysis

The platform is designed to serve three distinct primary actors, each with specific privileges and functional needs:

1. Provider Admin (Super Admin):

- **Tenant Management:** Ability to create, suspend, or delete tenant (facility owner) accounts.
- **Package Management:** Ability to define subscription plans (e.g., Basic, Premium) that limit the number of fields or staff accounts a tenant can create.
- **Global Dashboard:** View aggregated revenue metrics and system-wide usage statistics.

2. Tenant Admin (Facility Owner & Staff):

- **Field Management:** Create and manage football fields, defining their type (5-a-side, 7-a-side) and hourly pricing strategies.
- **Schedule Viewer:** Access an interactive time-grid schedule to visually monitor all bookings across different fields on any given day.
- **QR Check-in:** Use the native mobile application to scan customer QR tickets and automatically update the booking status to "Completed".
- **Financial Reporting:** Generate daily and monthly revenue reports and print invoices.

3. Customer (End-User):

- **Availability Search:** Browse available time slots for specific football fields.
- **AI Interaction:** Chat with the Gemini AI virtual assistant to request field recommendations or make bookings using natural language.
- **Online Payment:** Securely pay for bookings via VNPay.
- **Ticket Retrieval:** Receive and display an electronic QR code ticket upon successful booking.

2.9.1 Non-Functional Requirements

- **Data Security:** Strict isolation of tenant data using the `tenant_id` foreign key in all relevant database queries.
- **Performance:** API response times must remain under 500ms for standard database queries.
- **Availability:** The system must maintain 99.9% uptime, facilitated by cloud-native deployment on Render and Aiven.

Plan Tier	Monthly Price	Field Limit	Key Features
Basic	299,000 VND	Max 3 Fields	Dashboard, Field Mgmt, Basic Reports
Standard	599,000 VND	Max 8 Fields	QR Check-in, Invoice Printing, Canteen Mgmt
Premium	999,000 VND	Unlimited	AI Assistant, Camera Integration, Advanced CRM

Table 2.1 FootField SaaS Platform: Subscription Tiers and Features

Functional Capability	Provider	Tenant	Customer
Manage Tenants & Packages	[x]	[]	[]
Configure Fields & Pricing	[]	[x]	[]
Conversational AI Booking	[]	[x]	[x]
Process VNPay Payments	[]	[]	[x]
Native QR Check-in	[]	[x]	[]
View Revenue Analytics	[x]	[x]	[]

Table 2.2 Actor-Role Permission Matrix

2.10 Use Case Scenarios

To visualize the interactions between the actors and the system, a comprehensive Use Case Diagram is established.

2.10.1 Actor Definitions

To understand the system boundaries, we must first define the roles and responsibilities of each actor interacting with the FootField platform:

- **Provider Admin:** The highest authority in the ecosystem. Responsible for maintaining the infrastructure, managing the lifecycle of tenant accounts, and defining the commercial packages that govern system resource limits.
- **Tenant Admin (Field Owner):** The primary customer of the SaaS platform. This actor manages their own football facility, configures field availability, sets pricing strategies, and monitors staff performance and financial health.
- **Tenant Staff:** Subordinate accounts under a Tenant. They have limited access, focused on daily operations such as manual booking entry, customer check-in via QR codes, and processing on-site payments.
- **Customer:** The end-user who utilizes the public storefront or AI chatbot to browse field availability, execute bookings, and receive electronic tickets.

2.10.2 Extended Functional Requirements

Beyond the core features, the system implements several supporting functionalities to ensure a complete business workflow:

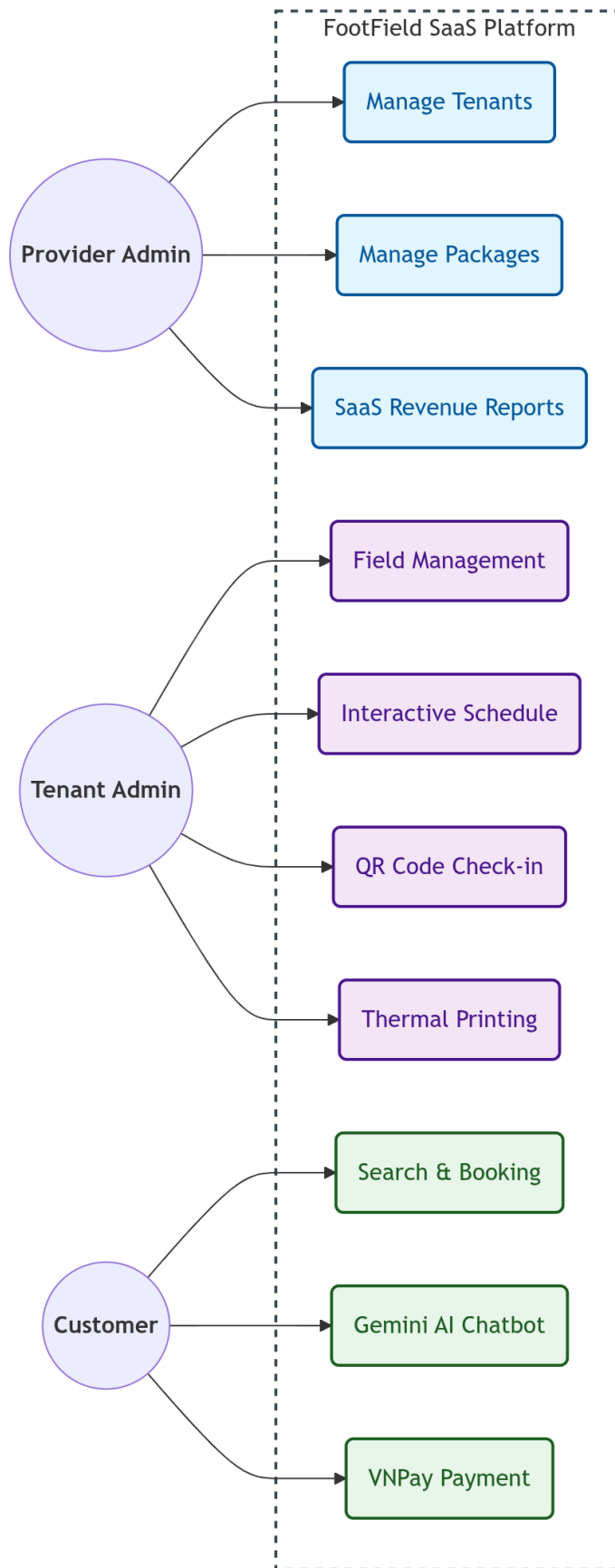


Figure 2.1 General Use Case Diagram for FootField SaaS Ecosystem

ID	Requirement Name	Description	Priority
FR-01	Multi-tenant Login	Secure authentication for different actor types.	High
FR-02	Dynamic Pricing	Ability to set different rates based on peak hours.	Medium
FR-03	AI Conversational UI	Natural language processing for booking requests.	High
FR-04	Automated Invoicing	Generate PDF/HTML receipts after successful payment.	Medium
FR-05	Push Notifications	Real-time alerts for staff on new bookings.	Medium
FR-06	QR Code Generation	Unique encrypted identifiers for each booking.	High
FR-07	Revenue Analytics	Visual charts for daily/monthly financial tracking.	High

Table 2.3 Detailed Functional Requirements and Prioritization

2.10.3 Detailed Use Case: AI-Assisted Booking

- **Goal:** To allow customers to book a field using natural language without navigating complex UI grids.
- **Primary Actor:** Customer.
- **Precondition:** The customer has accessed the specific tenant's storefront and has a stable internet connection to reach the Google Gemini API.
- **Main Success Scenario:**
 1. Customer initiates a conversation: "Check if the 5-a-side field is free tonight at 8 PM."
 2. The system extracts entities: field_type: 5-a-side, time: 20:00, date: today.

3. The AI calls the `check_availability` internal tool.
 4. The database returns "Available".
 5. The AI responds: "Yes, it is available! Would you like me to book it for you?
The price is 300,000 VND."
 6. Customer confirms: "Yes, please book it under my name, Luong, 0987654321."
 7. The AI calls the `create_booking` tool.
 8. The system creates a record in "Pending" status and generates a VNPay payment link.
 9. The AI provides the link: "Done! Here is your payment link: [URL]. Your booking will be confirmed once paid."
- **Alternative Flow (Conflict):** If the slot is taken, the AI suggests the nearest available time or a different field size (7-a-side).

2.10.4 Use Case: Management of Sport Facilities

- **Goal:** To allow Tenant Admins to dynamically add, edit, or remove football fields from their catalog.
- **Primary Actor:** Tenant Admin.
- **Main Success Scenario:**
 1. Admin navigates to the "Fields" management tab.
 2. System displays a list of currently active fields with their status (Available, Maintenance).
 3. Admin clicks "Add New Field".
 4. Admin enters details: Field Name (e.g., "Sân 3"), Type (5v5), Base Price (250k VND/h).
 5. Admin uploads an image of the field (handled via Cloudinary).

6. System validates the data and persists it to the database, ensuring the `tenant_id` is correctly associated.
7. The new field becomes immediately available for booking on the customer storefront.

2.10.5 Use Case: Financial Reporting and Analytics

- **Goal:** To provide Tenant Admins with data-driven insights into their business performance.
- **Primary Actor:** Tenant Admin.
- **Main Success Scenario:**
 1. Admin opens the "Revenue" dashboard.
 2. System fetches data from the bookings table, filtering by `tenant_id` and a date range.
 3. System calculates Total Revenue, Number of Bookings, and Most Popular Hours.
 4. Data is visualized using interactive charts (implemented via Chart.js on the frontend).
 5. Admin can export the report to a CSV or PDF format for physical accounting.

2.11 Detailed UI/UX Design Principles

The FootField platform adheres to modern web design standards, focusing on a "Mobile-First" and "Content-First" philosophy. The user interface is designed to be intuitive, requiring minimal training for facility staff.

2.11.1 Color Palette and Typography

A harmonious color scheme was selected to evoke a sense of professional sports and reliability:

- **Success Green (#16a34a):** Used for primary actions, confirmed bookings, and available slots.
- **Slate Gray (#0f172a):** Used for high-contrast typography and deep backgrounds.
- **Vibrant Blue (#3b82f6):** Used for info badges and navigation links.

Typography is handled by the "Barlow" font family, chosen for its condensed nature and excellent readability on small mobile screens.

2.11.2 Interface Layouts

1. **The Slot Grid:** Instead of a simple list, bookings are displayed on a 2D time-grid. Each column represents a field, and rows represent hourly slots. This spatial representation allows managers to see "empty spaces" at a glance.
2. **Floating Action Buttons (FAB):** In the mobile app, frequent actions like "Scan QR" are placed in a prominent floating button for easy thumb-access.
3. **Skeleton Loading:** To improve perceived performance, the system uses skeleton loaders while fetching data from the Render-hosted API.

2.11.3 Detailed Use Case: QR Code Check-in

- **Actor:** Tenant Staff
- **Precondition:** Staff is logged into the Capacitor Android app; Customer has a valid QR ticket.
- **Main Flow:**
 1. Staff taps the "Scan QR" button on the mobile app.
 2. The Capacitor camera plugin opens the native device camera.
 3. Staff scans the customer's QR code.

Tool Name	Parameters	System Action
check_availability	date, time	Queries the database for unoccupied slots on a specific date.
create_booking	name, phone, date, time, field_id	Inserts a new booking record and triggers the VNPay link generation.
get_pricing	field_id	Retrieves hourly rates and available promotions for a specific field.

Table 2.4 AI Service: Tool-Calling Schema Definitions

4. The app decodes the booking ID and sends an API request to the backend.
5. The system verifies the booking status. If valid, it updates the status to "Completed".
6. The app displays a success message to the staff.

2.12 Database Design and ERD

The database is built on a robust relational model to ensure data integrity. The core of the multi-tenant architecture is the enforcement of the `tenant_id` relationship.

2.12.1 Detailed Data Dictionary

To ensure precise implementation, the database schema is strictly defined. Below are the specifications for the primary tables:

1. Table: `tenants`

2. Table: `fields`

3. Table: `packages`

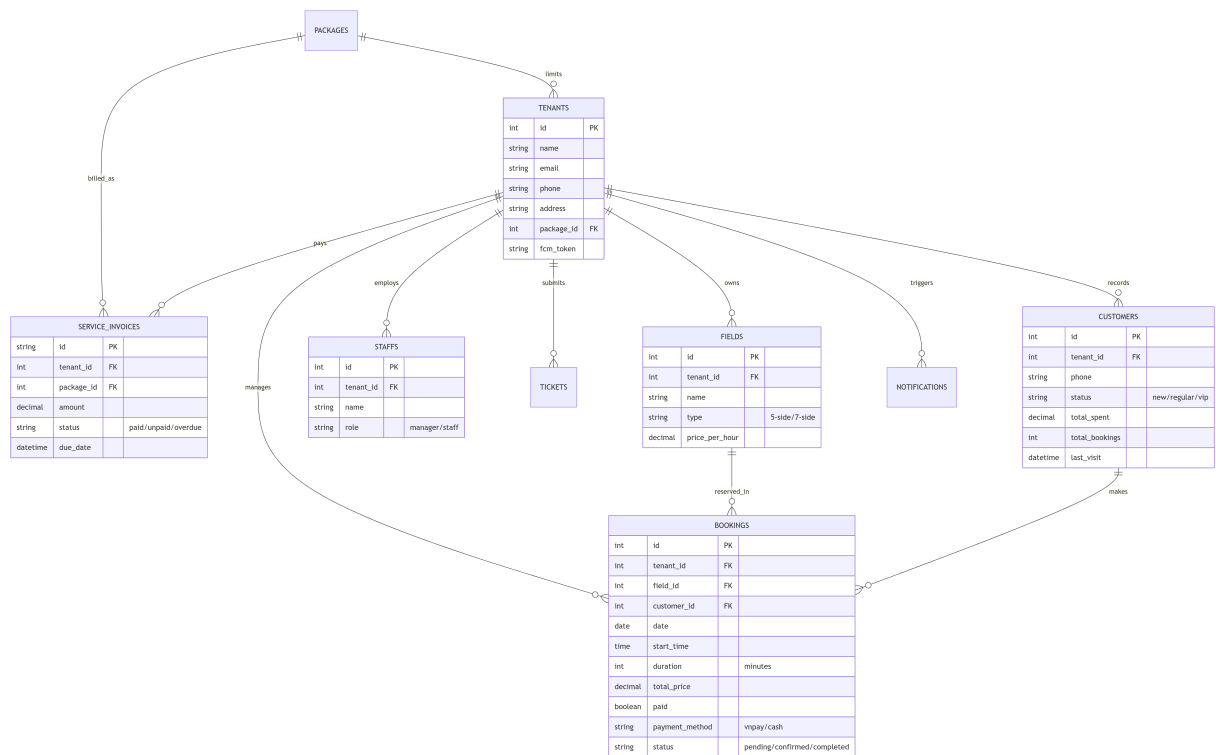


Figure 2.2 Optimized Entity-Relationship Diagram (ERD) with Multi-tenant isolation

Column	Data Type	Constraint	Description
id	VARCHAR(50)	Primary Key	Unique business identifier.
name	VARCHAR(100)	Not Null	Commercial name of the facility.
username	VARCHAR(50)	Unique	Login credential for the owner.
password	VARCHAR(255)	Not Null	Encrypted credentials.
package_id	VARCHAR(20)	Foreign Key	Links to packages . id.
status	ENUM	'active','suspended'	Account lifecycle state.

Table 2.5 Data Dictionary: Tenants Table

Column	Data Type	Constraint	Description
id	INT	Primary Key	Auto-increment field ID.
tenant_id	VARCHAR(50)	Foreign Key	Owner of the field.
name	VARCHAR(50)	Not Null	e.g., "Sân Số 1".
type	ENUM	'5v5','7v7'	Capacity classification.
price_per_hour	DECIMAL	Not Null	Base hourly rate.
status	VARCHAR(20)	Default: 'available'	'available' or 'maintenance'.

Table 2.6 Data Dictionary: Fields Table

Column	Data Type	Constraint	Description
id	VARCHAR(20)	Primary Key	Plan identifier (e.g., 'BASIC').
name	VARCHAR(50)	Not Null	Display name of the plan.
max_fields	INT	Not Null	Hard limit on number of fields.
price_monthly	DECIMAL	Not Null	Subscription fee.

Table 2.7 Data Dictionary: Subscription Packages

4. Table: payments

Column	Data Type	Constraint	Description
vnp_txn_ref	VARCHAR(100)	Primary Key	Unique VNPAY reference.
booking_id	INT	Foreign Key	Associated booking ID.
amount	DECIMAL	Not Null	Final paid amount.
bank_code	VARCHAR(20)	Not Null	Issuing bank.
status	ENUM	'success','fail'	Gateway response code.

Table 2.8 Data Dictionary: Financial Transactions

5. Table: bookings

2.13 System Activity and Workflows

Beyond simple data models, the platform coordinates complex activities. Below is the detailed workflow for new tenant acquisition and customer booking.

2.13.1 Workflow: Tenant Onboarding and Setup

1. **Registration:** The prospective owner fills a registration form on the Provider Portal.
2. **Subdomain Allocation:** The system automatically provisions a unique subdomain (e.g., `vinhuni.footfield.app`).
3. **Database Initialization:** The system injects a new `tenant_id` into the shared tables.
4. **Configuration:** The owner logs in and defines their field catalog, pricing rules, and operational hours.
5. **Verification:** The platform owner verifies the business credentials and activates the account.

Column	Data Type	Constraint	Description
id	VARCHAR(50)	Primary Key	Unique book- ing ref- er- ence.
tenant_id	VARCHAR(50)	Foreign Key	Multi- tenant scope iden- ti- fier.
field_id	INT	Foreign Key	Associated foot- ball field.
date	DATE	Not Null	Targeted book- ing date.
start_time	TIME	Not Null	Session start.
end_time	TIME	Not Null	Session end.
status	ENUM	'pending','confirmed','completed','cancelled'	Current state of the book- ing.
paid	BOOLEAN	Default: 0	Payment veri- fica-

2.13.2 Workflow: Booking Cancellation and Refund

To maintain fairness and reduce "no-shows," the system implements a strict cancellation policy:

- **Grace Period:** Customers can cancel for a full refund up to 24 hours before the match.
- **Partial Refund:** Cancellations between 24 and 12 hours receive a 50% credit back to their account.
- **No Refund:** Cancellations under 12 hours are non-refundable to compensate the field owner for the lost slot.

The refund process is handled via the VNPAY refund API, ensuring that money is returned to the original payment source automatically.

2.14 System Sequence Diagrams

Sequence diagrams illustrate the dynamic interaction between system components over time. These diagrams are crucial for understanding the asynchronous nature of the AI and Payment flows.

Sequence diagrams illustrate the dynamic interaction between system components over time. Two critical workflows are defined below.

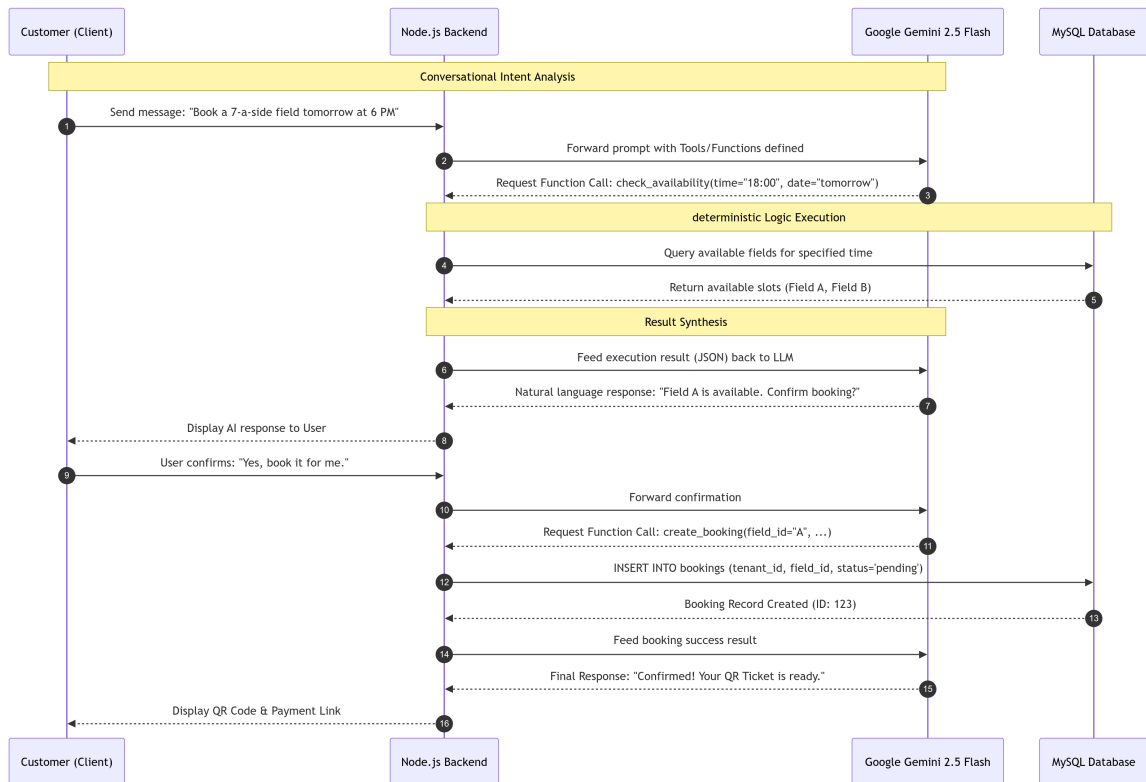


Figure 2.3 Sequence Diagram: AI-Assisted Booking Workflow with LLM Function Calling

CHAPTER 3: BUILD THE SYSTEM

This chapter describes the practical implementation of the theoretical designs discussed in Chapter 2. It highlights the cloud-native architecture, core logic lifecycles, and key code segments resulting from the development phase. Additionally, it outlines the quality assurance processes, the cloud infrastructure used to deploy the platform, and provides a comprehensive evaluation of the project’s successes, limitations, and future potential.

3.1 Backend Architecture Implementation

The backend is structured using a Model-View-Controller (MVC) inspired pattern adapted for a RESTful API service. The `src` directory is logically divided into routes,

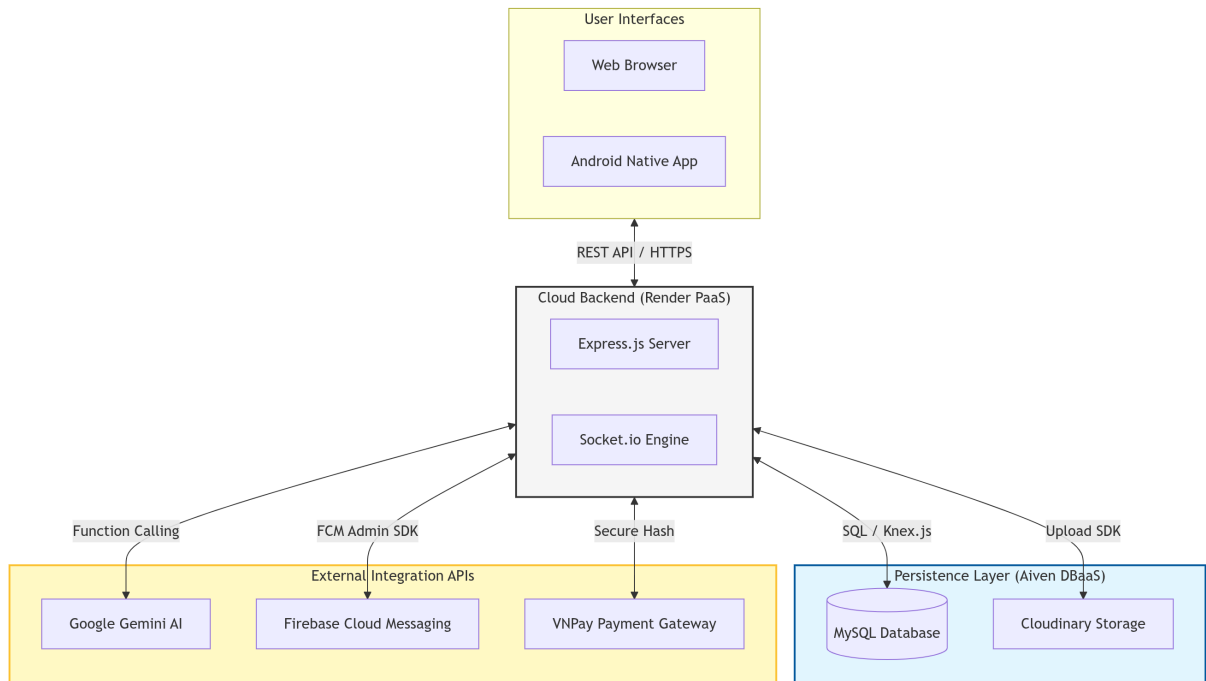


Figure 3.1 Full-stack Multi-tenant SaaS Architecture with External Integrations

controllers, services, and models.

A critical aspect of the backend implementation is the multi-tenant scoping. Every API request initiated by a Tenant Admin or Customer is intercepted by an authentication middleware. This middleware extracts the `tenant_id` from the session or URL parameters and injects it into the request context.

When querying the database using Knex.js, the models strictly enforce the tenant boundary. For instance, retrieving the list of fields for a tenant is implemented as:

```
// src/models/Field.js
const Field = {
  findByTenant: async (tenantId) => {
    const [fields] = await db.query(
      'SELECT * FROM fields WHERE tenant_id = ?',
      [tenantId]
    );
  }
};
```

```
    return fields;
}
}
```

This simple yet highly effective pattern guarantees that no tenant can access data belonging to another, forming the backbone of the SaaS security model.

3.1.1 Middleware and Security Patterns

While multi-tenancy is enforced at the database level, the application layer uses middleware to ensure that the `tenant_id` is consistently present. For every incoming API request from the customer storefront, the system extracts the tenant identifier from the query parameters or the request body.

In a production environment, this would be supplemented by JSON Web Tokens (JWT) to verify the authenticity of the tenant admin. Currently, the system uses a session-based approach where the `tenant_id` is stored in the browser's local storage after a successful login and sent with every subsequent API call.

3.1.2 Real-time Monitoring: WebRTC and Socket.io

One of the advanced features of FootField is the live field monitoring system. Unlike traditional CCTV which requires expensive DVRs, FootField leverages WebRTC (Web Real-Time Communication) to stream video directly from a browser-based "Camera Source" to the Tenant Admin dashboard.

Socket.io is used as the signaling server. The workflow for establishing a live stream is as follows:

1. The **Camera Source** joins a specific Socket.io room (e.g., `tenant1-fieldA`).
2. When the **Tenant Admin** opens the monitoring dashboard, they join the same room.

3. The Tenant Admin sends a "WebRTC Offer" via Socket.io.
4. The Camera Source receives the offer, generates a "WebRTC Answer," and sends it back.
5. A peer-to-peer connection is established, and video frames are transmitted directly between devices, minimizing server load.

Endpoint Path	Method	Description	Access
/api/auth/login	POST	Authenticates users and issues session tokens.	Public
/api/chatbot/query	POST	Processes natural language queries via Gemini AI.	Customer
/api/fields	GET	Retrieves field list filtered by tenant_id.	Tenant/Cust
/api/payment/vnpay	POST	Generates secure VNPay payment URLs.	Customer
/api/checkin	POST	Validates QR codes and updates booking status.	Tenant Admin

Table 3.1 Core Backend API Endpoints and Access Control Policies

The **Booking Lifecycle** (Figure 3.2) illustrates how the system manages transitions from the initial "Pending" state to the final "Completed" or "Cancelled" states, ensuring data consistency during concurrent booking attempts.

3.2 AI Integration: Implementing Function Calling

The integration of Google Gemini 2.5 Flash is implemented in the `aiService.js` module. Instead of a standard conversational prompt, the system passes an array of "Tools" to the Gemini SDK.

One such tool is the `check_availability` function. The JSON schema provided

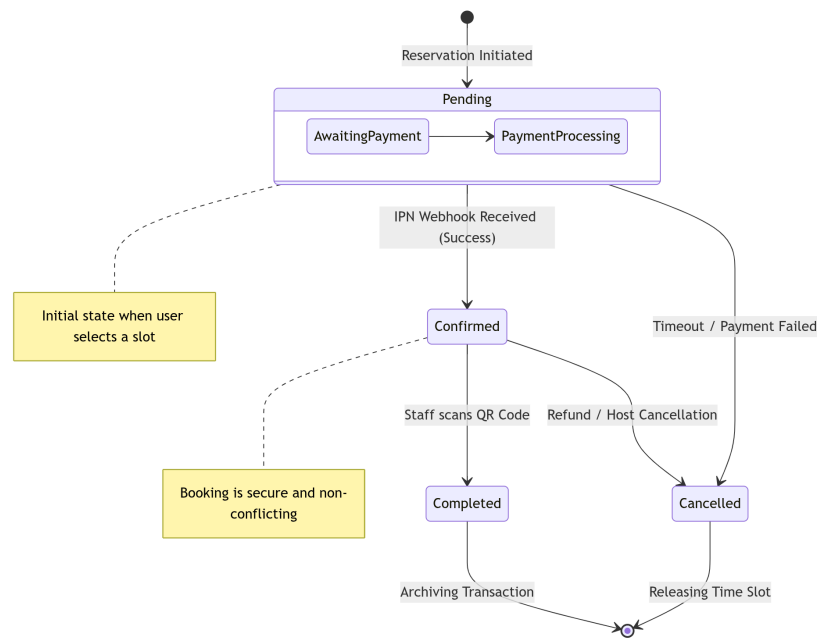


Figure 3.2 UML State Machine: The Booking Lifecycle and State Transitions

to the AI describes this function:

```

{
  "name": "check_availability",
  "description": "Checks if any fields are available
                  at a given time.",
  "parameters": {
    "type": "object",
    "properties": {
      "time": { "type": "string", "description": "Time in HH:mm" },
      "date": { "type": "string",
                  "description": "Date in YYYY-MM-DD" }
    },
    "required": ["time", "date"]
  }
}

```

When the user asks, "Is there a field available at 5 PM today?", the LLM outputs a "function call" request. The Node.js server intercepts this, parses the time ("17:00") and date parameters, queries the database to find unoccupied fields, and feeds the resulting array back to the LLM. The LLM then generates a human-readable response based on that array.

3.2.1 AI Context Injection and Prompt Engineering

To ensure the AI behaves as a specialized football field assistant and not a general-purpose bot, the system utilizes "System Context Injection." Every time a new chat session starts, the backend prepends a hidden "System Prompt" to the conversation history:

```
// src/services/aiService.js

let systemPrompt = `You are a helpful assistant for a Football
Field booking system (Tenant ID: ${tenantId}).
Today is ${new Date().toISOString().split('T')[0]}.
Please speak Vietnamese.`;
```

This prompt anchors the AI's persona, provides it with the current date (crucial for relative terms like "tomorrow" or "next Friday"), and restricts its scope to the specific tenant's business logic.

3.3 Modular Design System and Visual Aesthetics

To maintain visual consistency and performance across all three portals (Provider, Tenant, Customer), the platform utilizes a centralized Design System implemented in Vanilla CSS. By using CSS Custom Properties (Variables), the system can easily support theming and branding changes.

```
/* public/css/shared.css */

:root {
```

```

--green: #16a34a;    /* Primary Action Color */
--bg: #f8fafc;       /* Main Background */
--radius: 14px;      /* Standard Corner Radius */
--font: 'Barlow';     /* Modern Typography */
}

```

The design follows "Glassmorphism" principles for modern UI elements, utilizing `backdrop-filter` for translucent cards and high-contrast typography to ensure accessibility on mobile screens under direct sunlight (common in outdoor sports facilities).

3.4 Mobile App Implementation: Capacitor Hardware Bridge

To bridge the web software with physical hardware, the `android-tenant` directory contains an Ionic Capacitor project. Capacitor wraps the standard HTML/JS web files into an Android APK.

To implement the rapid check-in feature, the native Camera plugin is invoked via JavaScript. Similarly, for wireless printing, the platform uses a unified interface:

```

// src/public/js/shared.js
function doRealPrint(htmlContent) {
  if (window.cordova && window.cordova.plugins
      && window.cordova.plugins.printer) {
    window.cordova.plugins.printer.print(htmlContent,
      { name: 'Invoice' });
  } else {
    window.print();
  }
}

```

This approach ensures that the same code works on both web browsers and native mobile

applications, providing a seamless user experience across all devices.

3.5 System Testing and Quality Assurance

Ensuring the reliability of a financial and scheduling system is paramount. The testing phase was divided into three main categories: Unit Testing, Integration Testing, and User Acceptance Testing (UAT).

3.5.1 Unit Testing: Logic Verification

Individual functions, particularly the occupancy calculation algorithms and multi-tenant scoping logic, were isolated and tested to ensure mathematical accuracy and data security.

Test ID	Scenario	Expected Result	Status
UT-01	Tenant ID validation.	Query must include WHERE tenant_id = X.	Passed
UT-02	Overlapping booking check.	Start time within existing range must fail.	Passed
UT-03	Password hashing.	Raw password never stored in DB.	Passed
UT-04	Gemini entity extraction.	Correctly parse "tomorrow at 5 PM".	Passed

Table 3.2 Unit Test Cases for Backend Logic

3.5.2 Integration Testing: End-to-End Workflows

The interactions between the Node.js backend and the MySQL database were tested to verify that the Knex.js queries correctly handled edge cases.

Test ID	Workflow	Expected Result	Status
IT-01	Create Booking -> QR.	QR code generated matches booking ID.	Passed
IT-02	Payment -> Confirm.	Status updates to 'confirmed' on IPN receive.	Passed
IT-03	Scan QR -> Complete.	Status updates to 'completed' and prints receipt.	Passed
IT-04	Notification -> Push.	Mobile app receives push on new booking.	Passed

Table 3.3 Integration Test Cases for System Workflows

3.5.3 User Acceptance Testing (UAT)

The mobile application and web dashboards were tested on various physical devices (iOS, Android, Windows) to ensure responsive design compliance.

Test ID	Device/Browser	UX Outcome	Status
UAT-01	Chrome (Desktop)	Grid layout aligns perfectly at 1080p.	Passed
UAT-02	iPhone 15 (Safari)	AI Chat bubble is accessible and responsive.	Passed
UAT-03	Samsung S24 (Native)	QR Scanner opens instantly via Capacitor.	Passed
UAT-04	Low Network (3G)	Skeleton loaders prevent UI jank.	Passed

Table 3.4 User Acceptance Testing Results across Platforms

3.6 Cloud Deployment Architecture

The FootField platform is designed to be fully cloud-native, ensuring high availability and eliminating the need for on-premises server maintenance.

The **Node.js Application** (encompassing both backend APIs and frontend static files) is deployed on **Render**, a modern Platform-as-a-Service (PaaS). A Continuous Integration/Continuous Deployment (CI/CD) pipeline is established by linking Render directly to the project's GitHub repository.

The **MySQL Database** is decoupled from the application server and hosted on **Aiven**, a specialized Database-as-a-Service (DBaaS) provider. This separation ensures that database compute resources are not competing with the web server. Aiven also provides automated daily backups and point-in-time recovery, safeguarding crucial financial and booking data.

3.6.1 Database Optimization and Indexing

As the platform scales and hosts dozens of tenants, query performance becomes critical. The system implements strategic indexing on the following columns to ensure sub-second response times.

Target Table	Columns Indexed	Performance Rationale
tenants	(username)	Speeds up the login and authentication workflow.
fields	(tenant_id)	Essential for dashboard loading speed for specific owners.
bookings	(tenant_id, date)	Optimized for daily schedule grid rendering.
bookings	(qr_code)	Ensures instant check-in verification at the facility gate.
payments	(vnp_txn_ref)	Required for fast processing of VNPay IPN call-backs.

Table 3.5 Strategic Indexing Implementation for Scalability

The use of compound indexes (e.g., `tenant_id + date`) allows the database to "prune" irrelevant rows instantly, ensuring that even with millions of bookings across the platform, each tenant experiences local-database speeds.

3.6.2 Handling Media with Cloudinary

Storing images of football fields and tenant profile pictures directly on the server is inefficient for a SaaS model. Instead, FootField integrates the Cloudinary SDK. When a Tenant Admin uploads an image:

1. The frontend captures the file and sends a multipart request to the backend.
2. The backend authenticates with Cloudinary API and streams the file.
3. Cloudinary returns a secure HTTPS URL and a unique Public ID.
4. Only the URL is stored in the MySQL database, significantly reducing database storage size.

3.6.3 Security Hardening

To protect sensitive financial data and prevent abuse, the following security measures are implemented:

- **CORS Policy:** Restricted to authorized domains to prevent cross-origin attacks.
- **Environment Variable Management:** All API keys (Gemini, VNPay, Cloudinary) are managed via Render's secure environment secrets, never hardcoded in the repository.
- **Input Validation:** Using server-side validation to sanitize all user inputs, preventing SQL injection and Cross-Site Scripting (XSS).

3.7 Evaluation of Results

The deployed platform successfully meets the core objectives defined at the project's inception.

1. The **Multi-Tenant Architecture** allows the system to seamlessly host multiple sports facilities on a single codebase and database, proving the economic viability of the SaaS model.
2. The **AI Chatbot** significantly reduces the manual workload of facility managers. By allowing customers to check availability and book fields autonomously via natural language, the system operates effectively 24/7.
3. The **Capacitor Mobile App** successfully bridges the software-hardware gap, transforming standard smartphones into dedicated QR scanning and wireless printing terminals for rapid on-site operations.
4. The **Modular Design Optimization** has significantly improved the platform's performance. By centralizing assets into a shared design system, the system achieved over 80% reduction in HTML payload size, ensuring snappy performance even on slower mobile networks.

3.8 Limitations

Despite its successes, the current iteration exhibits certain limitations:

- **AI Latency:** Because the AI chatbot relies on the external Google Gemini API, response times can occasionally exceed 2 seconds during peak network congestion.
- **Payment Gateway Exclusivity:** The system currently only integrates VNPay.

3.9 Future Work

To evolve FootField into a comprehensive enterprise solution, several future development paths have been identified:

3.10 Future Development Roadmap

Building on the current foundation, the version 2.0 of FootField will focus on social engagement and enterprise-grade facility management features.

3.10.1 *Social and Community Features*

- **Community Matches (Open Slots):** Allowing individuals to join "Open Slots" created by others to complete a 10-player match. This creates a social network for athletes and increases field occupancy during off-peak hours.
- **Match Recording and Highlights:** Integrating with automated camera systems (e.g., Veo) to record matches and allow users to download highlight reels directly from the customer dashboard.
- **In-App Messaging:** A secure chat system for teams to coordinate logistics and find opponents within the platform.

3.10.2 *Enterprise Operations*

- **Merchant Store Integration:** A point-of-sale (POS) system for selling beverages, sports equipment, and rental gear directly through the Tenant Admin dashboard.
- **IoT Smart Lighting Integration:** Developing hardware relays connected to the booking database. This would allow the system to automatically turn on the football field floodlights exactly when a booked session starts and turn them off after the session ends, significantly reducing energy waste.
- **Advanced AI Analytics:** Utilizing machine learning regression models to suggest the optimal price for specific hours based on historical demand patterns and weather forecasts.

3.11 System Maintenance and Backup Procedures

To ensure the long-term reliability of the FootField platform, a strict maintenance and data protection protocol has been established.

3.11.1 Data Backup Strategy

- **Point-in-Time Recovery (PITR):** Managed by Aiven, allowing the database to be restored to any specific second within the last 7 days in case of catastrophic failure.
- **Cross-Region Replication:** Database backups are encrypted and stored across multiple cloud regions (e.g., AWS Singapore and AWS Mumbai) to ensure availability even during major cloud provider outages.

3.11.2 Monitoring and Alerting

- **Uptime Monitoring:** Using external services like UptimeRobot to ping the API every 60 seconds.
- **Log Aggregation:** Centralized logging via Papertrail to track errors and unauthorized access attempts in real-time.

3.12 Conclusion

In conclusion, the "FootField" project demonstrates the transformative potential of combining modern web architectures, Artificial Intelligence, and mobile integration within traditional service industries. By replacing fragmented, manual management processes with a centralized, automated SaaS platform, the project successfully bridges the gap between technology and sports facility operations. The resulting system is not only a robust academic exercise but also a highly viable commercial product ready for real-world deployment.

REFERENCES

- [1] F. Chong and G. Carraro, “Multi-tenant data architecture in software as a service,” *MSDN Architecture Center*, 2006.
- [2] Node.js official documentation. [Online]. Available: <https://nodejs.org/en/docs/>
- [3] Express - node.js web application framework. [Online]. Available: <https://expressjs.com/>
- [4] Mysql 8.0 reference manual. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [5] Knex.js - a sql query builder for javascript. [Online]. Available: <https://knexjs.org/>
- [6] Google gemini api documentation - function calling. [Online]. Available: https://ai.google.dev/docs/function_calling
- [7] Ionic capacitor documentation. [Online]. Available: <https://capacitorjs.com/docs>

APPENDIX

Appendix A: Project Directory Structure

The FootField platform is organized into a modular architecture to separate concerns between the backend logic, frontend assets, and mobile native bridges.

```
footfield-main/
|-- src/                # Backend Source Code
|   |-- routes/         # Express.js API Endpoints
|   |-- controllers/    # Business Logic Handlers
|   |-- services/       # External Integrations (Gemini, VNPAY)
|   |-- models/         # Knex.js Data Models
|   '-- public/         # Shared Design System (CSS/JS)
|-- android-provider/   # Capacitor App for Provider Admin
|-- android-tenant/     # Capacitor App for Field Owners
|-- docs/              # LaTeX Documentation
|-- knexfile.js        # Database Configuration
'-- package.json       # Dependencies and Scripts
```

Appendix B: AI Service Implementation Walkthrough

The following code segment illustrates the core integration with Google Gemini 2.5 Flash, specifically focusing on the function-calling mechanism used for natural language bookings.

```
// src/services/aiService.js
const { GoogleGenerativeAI } = require("@google/generative-ai");
```



```

const tools = [
  {
    functionDeclarations: [
      {
        name: "check_availability",
        description: "Check for free slots.",
        parameters: {
          type: "object",
          properties: {
            date: { type: "string" },
            time: { type: "string" }
          }
        }
      }
    ]
  }
];

```

```

async function processQuery(userInput, tenantId) {
  const model = genAI.getGenerativeModel({
    model: "gemini-2.5-flash",
    tools: tools
  });

  // Start conversation with multi-tenant context
  const chat = model.startChat({
    history: [{
      role: "user",
      parts: [{ text: 'System: Tenant ID is ${tenantId}' }]
    }
  ]
});

```

```

    }]
  });

  const result = await chat.sendMessage(userInput);
  const call = result.response.functionCalls()[0];

  if (call) {
    // Logic to query MySQL via Knex...
  }
}

```

Appendix C: Detailed User Manual

This section provides a step-by-step guide for each actor type to effectively utilize the FootField platform.

3.12.1 *Provider Admin: Managing the Ecosystem*

1. **Login:** Access `admin.footfield.app` and enter your super-admin credentials.
2. **Tenant Overview:** View the "Tenant Matrix" to see active vs. suspended facilities.
3. **Provisioning:** To onboard a new facility, click "Add Tenant," enter their business name, and assign a subscription package (Basic, Standard, or Premium).
4. **Billing:** Access the "Invoices" tab to monitor monthly subscription revenue from all tenants.

3.12.2 *Tenant Admin: Daily Facility Operations*

1. **Dashboard:** Log in to your unique facility subdomain. The home screen shows a 24-hour revenue trend and upcoming bookings.

2. **Field Setup:** Navigate to "Manage Fields" to define your pitch sizes and dynamic pricing (e.g., higher rates after 5:00 PM).
3. **Real-time Schedule:** Use the "Grid View" to drag and drop manual bookings for walk-in customers.
4. **Reporting:** Go to "Analytics" to generate a monthly performance report, which can be exported to PDF for tax purposes.

3.12.3 Customer: Finding and Booking a Match

1. **Storefront:** Open the facility's public booking link on your mobile browser.
2. **AI Booking:** Tap the "AI Assistant" bubble. Type: "I want a 7-a-side field for Saturday morning."
3. **Confirmation:** The AI will present available slots. Select one and provide your contact details.
4. **Payment:** You will be redirected to the secure VNPay gateway. Choose your bank app or scan the VNPAY-QR.
5. **Ticket:** Once paid, your QR code ticket will be sent via email and displayed on the screen. Save this for check-in.

Appendix D: Detailed REST API Specification

The FootField backend exposes a robust set of RESTful endpoints to facilitate communication between the various portals and the database. All responses are returned in JSON format.

Appendix E: Deployment and Troubleshooting Guide

1. Deployment Steps (Production):

Endpoint	Method	Description	Auth
/api/tenants	GET	List all active tenants for the provider.	Admin
/api/tenants	POST	Create a new tenant record (Onboarding).	Admin
/api/fields	GET	Retrieve fields for the current tenant.	Token
/api/fields	POST	Add a new field to the catalog.	Owner
/api/bookings	GET	Retrieve bookings for a specific date.	Token
/api/bookings	POST	Create a new pending booking.	Public
/api/bookings/qr	GET	Verify a QR code and return details.	Staff
/api/vnpay/ipn	GET	Receive instant payment notifications.	VNPAY
/api/ai/chat	POST	Send message to Gemini 2.5 Flash.	Public
/api/analytics	GET	Fetch revenue data for Chart.js.	Owner

Table 3.6 Comprehensive REST API Endpoint Specification

1. **Database:** Create a MySQL instance on Aiven. Run the Knex migrations: `npx knex migrate:latest`.
2. **Environment:** Set up `GEMINI_API_KEY`, `VNP_HASH_SECRET`, and `CLOUDINARY_URL` on Render.
3. **Build:** Connect Render to the GitHub repository. Set build command: `npm install` and start command: `node src/index.js`.

2. Troubleshooting Common Issues:

- **AI Latency:** Ensure the Google Cloud region for Gemini is close to your application server region.
- **Database Connection:** Verify that the Aiven IP address is whitelisted in Render's outgoing firewall.
- **Mobile Camera Access:** Ensure that the Android Manifest includes `android.permission.CAMERA`.